Sports-v1 .NET Testing Study Guide

Date: April 24, 2026

> Reviewed from current code in `dai/platform/dotnet/DevCore.Api.Tests`, `DevCore.Api`, and the current sports-v1 vault docs.
>
> Scope: the .NET test setup around the sports-v1 artifact pipeline and the `AgentRunsController` failure-persistence path.

Executive Summary

The current sports-v1 .NET test strategy is intentionally narrow. It uses one xUnit test project, mostly real instances for pure logic, hand-written fakes at I/O boundaries, and one small host-level integration slice for controller failure persistence.

As of April 24, 2026, the suite contains 32 passing tests:

- 30 unit tests around the artifact pipeline and orchestration logic
- 2 host-level integration tests for `AgentRunsController`

This setup is designed to protect the main risks in the current slice without expanding into a broad end-to-end or infrastructure-heavy test architecture.

Current Testing Stack

Area	Current choice	Why it matters
test framework	`xUnit`	simple, familiar, and already aligned with the existing test project
test runner	`dotnet test`	standard local run path; no custom harness
test sdk	`Microsoft.NET.Test.Sdk`	normal .NET test discovery and execution
integration host	`Microsoft.AspNetCore.Mvc.Testing` with `WebApplicationFactory<Program>`	spins up the ASP.NET Core app host without building a separate custom test server
test database for host slice	`Microsoft.EntityFrameworkCore.InMemory`	lightweight persistence verification for controller host behavior
mocking style	no mocking framework	keeps the slice explicit; fakes are hand-written and easy to read
visibility for artifact state tests	`InternalsVisibleTo("DevCore.Api.Tests")` in `DevCore.Api.csproj`	allows tests to call internal `Record*` methods on `SportsRunArtifact` directly

The test project is `dai/platform/dotnet/DevCore.Api.Tests/DevCore.Api.Tests.csproj` and targets `net10.0`.

Why This Approach Was Chosen

The current pipeline splits cleanly into two kinds of code:

1. Pure computation
2. I/O and host wiring

That split drives the testing choices.

For pure computation, the code uses real instances because there is no reason to fake them. `SportsEvaluator` and `SportsComposer` are deterministic and side-effect free, so the tests can instantiate them directly and assert exact state transitions and outputs.

For I/O boundaries, the tests use small hand-written fakes instead of a mocking framework. That keeps the scenarios obvious. In `AgentRunServiceTests`, for example, the collector and analyzer are replaced with tiny success, failure, and cancellation fakes so the tests can prove orchestration behavior without HTTP, FastAPI, or model calls.

The integration slice exists for one specific gap that unit tests do not cover: the controller's durable side effect when the service throws `AnalysisPipelineException`. Unit tests can prove the exception and failure artifact exist. The host-level test proves the ASP.NET Core app actually persists the failed `AgentRun` row correctly.

Current Test Inventory

Test class	File	Count	Main concern
`SportsRunArtifactTests`	`AgentRuns/SportsRunArtifactTests.cs`	7	artifact state transitions and pipeline metadata recording
`SportsEvaluatorTests`	`AgentRuns/SportsEvaluatorTests.cs`	7	confidence calibration, confidence bands, and evaluate-step recording
`SportsComposerTests`	`AgentRuns/SportsComposerTests.cs`	10	final result composition on both success and analyze-failure paths
`AgentRunServiceTests`	`AgentRuns/AgentRunServiceTests.cs`	6	pipeline orchestration, failure wrapping, and cancellation behavior
`AgentRunsControllerTests`	`Integration/AgentRunsControllerTests.cs`	2	host-level persistence when `IAgentRunService` throws `AnalysisPipelineException`

Total: 32 tests.

How the Main Test Groups Work

`SportsRunArtifactTests`

These are the lowest-level state-machine tests. They create a `SportsRunArtifact`, call internal `Record*` methods directly, and inspect the mutable artifact after each call.

This is where the test suite proves details such as:

- degraded collect results downgrade publishability to `PublishableWithCaveats`
- degradation notes are appended when expected
- `RecordAnalyzeFailed` adds a failed `analyze` step
- `RecordAnalyzeFailed` makes the artifact `NotPublishable`

Mechanically, this is possible because `DevCore.Api.csproj` exposes internals to the test assembly.

`SportsEvaluatorTests`

These tests use a real `SportsEvaluator` and build artifacts in carefully chosen grounding states before calling `Evaluate`.

The arrange/act/assert shape is consistent:

1. arrange an artifact with a known competition and known grounded-signal richness
2. act by calling `Evaluate`
3. assert calibrated confidence, confidence band, and the recorded `evaluate` step

The tests cover the current calibration tiers:

- priors-only runs
- partially grounded runs
- fully grounded runs

They also verify that evaluator output is the calibrated confidence used later by composition.

`SportsComposerTests`

These tests use a real `SportsComposer` and feed it either:

- an artifact that already has collect, analyze, and evaluate results
- or an artifact that has a failed analyze stage

They prove that compose behavior is not just "summary assembly". It also carries forward observability and learning-loop metadata. Important assertions include:

- final confidence comes from the evaluator, not directly from the analyzer
- analyzer confidence is still preserved separately
- pipeline steps are carried into the final result in order
- the analyze-failure path produces a minimal not-publishable artifact with a skipped compose step

`AgentRunServiceTests`

These are orchestration tests for the service layer, not host tests. The service is built with:

- real `SportsEvaluator`
- real `SportsComposer`
- fake collector and analyzer implementations chosen per scenario

This is the clearest example of the repo's current testing style: use real logic where possible, and fake only the boundaries that would otherwise introduce I/O or nondeterminism.

The tests prove:

- analyze failures are wrapped as `AnalysisPipelineException`
- the wrapper carries a non-null failure artifact
- the failure artifact is already marked `NotPublishable`
- the failed `analyze` step is present in pipeline metadata
- cancellation is not wrapped
- the success path returns a result with all four pipeline steps

What Each Test Group Protects

Test group	What it protects today
artifact tests	regressions in publishability state, degradation-note behavior, and step recording
evaluator tests	regressions in confidence dampening, clamping, and confidence-band labeling
composer tests	regressions in final artifact shape, metadata carry-through, and failure-artifact composition
service tests	regressions in pipeline sequencing, failure wrapping, and cancellation behavior
controller integration tests	regressions in durable failure persistence after the ASP.NET Core host receives a thrown `AnalysisPipelineException`

The current suite is strongest around failure-state correctness and artifact integrity. That matches the current slice risk: a failed analyze stage should still leave behind a durable, inspectable, metadata-rich run artifact.

Integration Test Path

The integration test class is `DevCore.Api.Tests.Integration.AgentRunsControllerTests`.

Its host setup is intentionally light:

- `WebApplicationFactory<Program>` boots the real ASP.NET Core app
- `Program` exposes `public partial class Program { }` so `WebApplicationFactory<Program>` can bind to the top-level app entry point
- the existing `AppDbContext` registration is removed and replaced with EF Core InMemory
- `IIdentityResolver` is replaced with a fake resolver
- `IAgentRunService` is replaced with a per-test stub
- logging providers are cleared to keep the test host simple and avoid host-specific sink issues

The in-memory provider is being used here only to verify host behavior and persistence wiring. It is not meant to prove relational database semantics, SQL Server defaults, constraints, or query-plan behavior.

The important controller path under test is:

```
POST /api/agent-runs
  -> controller creates pending AgentRun row
  -> stubbed IAgentRunService throws AnalysisPipelineException
  -> controller catches it, marks the row failed, stores FailureArtifact in OutputJson,
     copies the original inner error message into ErrorMessage, saves, then rethrows
```

The tests then inspect the stored `AgentRun` row and prove that:

- `Status` becomes `failed`
- `OutputJson` contains the serialized failure artifact, not the initial `{}`
- publishability metadata is preserved

- pipeline step metadata is preserved
- `ErrorMessage` stores the original stage error, not the wrapper message

One practical detail matters here: `TestServer` rethrows the unhandled exception instead of returning a production-style HTTP 500 payload through the normal deployed middleware path. That is why the test asserts the exception and then checks database state.

What Is Deliberately Out Of Scope

The current slice does not try to prove everything. It intentionally does not cover:

- FastAPI behavior
- real model calls
- external sports data APIs
- JWT authentication wiring end to end
- SQL Server-specific constraints, defaults, indexes, or provider behavior
- a broad success-path controller integration suite
- frontend behavior
- full end-to-end system tests

That is deliberate. The current suite is a maintainable slice, not a full-system testing framework.

How To Run The Tests

From `dai/platform/dotnet`:

Run all tests in the test project:

```
dotnet test .\DevCore.Api.Tests\DevCore.Api.Tests.csproj --no-restore -m:1
```

Run only the controller integration test class:

```
dotnet test .\DevCore.Api.Tests\DevCore.Api.Tests.csproj --no-restore -m:1 --filter
"FullyQualifiedName~DevCore.Api.Tests.Integration.AgentRunsControllerTests"
```

Verified locally on April 24, 2026:

- all tests: 32 passed
- controller integration class: 2 passed

Practical Notes And Caveats

- `-m:1` is currently the safest local run mode in this workspace because it avoids noisy parallel build behavior across referenced projects.
- The EF Core InMemory provider in `AgentRunsControllerTests` is only a host-behavior tool. If the goal later becomes proving relational semantics, the provider choice should be revisited.

- `SuppressTfmSupportBuildWarnings` is set in `DevCore.Api.csproj` and `DevCore.Data.csproj` because EF design-time tooling pulls `mono.texttemplating`, which warns on `net10.0` and otherwise interferes with local test builds.
- The project still uses code as the source of truth for the pipeline contract. One older vault contract snippet lags the current sports request/context shape, so code should win if there is any disagreement.
- There is no mocking framework in this slice. That keeps tests readable, but it also means new I/O scenarios should stay small and explicit rather than accumulating a large fake infrastructure layer.

Current State Summary

The sports-v1 .NET test setup is in a healthy narrow state:

- one test project
- real instances for pure computation
- hand-written fakes at boundaries
- focused unit tests around artifact mechanics and orchestration
- one narrow host-level integration slice for controller failure persistence
- no FastAPI involvement
- no real model calls

That is the right shape for the current maturity of the slice. The next expansion should only happen when a new failure mode or confidence-critical behavior cannot be covered cleanly by the existing unit tests and the current single integration host path.